Grazioso Salvare Animal Rescue Dashboard

Aaron Vogel

October 11, 2026

My artifact is the Grazioso Salvare animal rescue dashboard that I built for CS 340 in 2026. It's a Python and Dash app which uses the aac.animals collection in MongoDB to filter the records, provide breed summaries and show animal locations. The original project included a Python CRUD module and a Jupyter notebook for the dashboard.

I picked this one because it's a good example of database work being applied to a real world scenario. Added schema validation to MongoDB and a compound index on the rescue filters. Moved all the filtering, sorting and pagination into MongoDB and added an aggregation to count the breeds. The CRUD module pulls its connection string from an env var, escapes the breed search text, limits the sort fields and page sizes and requires a specific filter for the single record updates and deletes. There are 11 unit tests for the database behaviour and error handling and a benchmark script which compares a forced collection scan against a query using the compound index.

My Module One plan was to deal with Outcomes 2, 3, 4 and 5. The README and this narrative are the bits that explain the work to the reader. The query/index/pagination changes are the bits that I can use to assess the design decisions and the trade-offs that have been made. Schema validation and aggregation are the bits that demonstrate database skills and the environment based credentials and restricted writes are the bits that support the security outcome. That outcome coverage has been retained in the final version.

One of the problems we found with the original dashboard was that it could display 10 records and still be downloading the entire collection. In the 10 record view the dashboard now requests 10 records per page rather than downloading the entire collection which means far fewer records are being sent to the dashboard per page. Moving the pagination to MongoDB required some changes to the callbacks and I've checked that the table, chart and map still work in concert. I also learned that indexes should match the queries that they support as they do use storage and add a bit of work to database writes. Security and error handling were also key. The hard coded password has been removed and the connection string is now in an environment variable. The error handling has been changed so that a valid search which returns no matches is not mistaken for a database failure. Updates and deletes require a specific filter to help prevent accidental changes. We were unable to run the benchmark against the live AAC database as the source ZIP didn't contain the AAC data and there was no live MongoDB service available. For that reason we can't actually report any speed gain from the index. The benchmark script can be run once the AAC data is available.